

LCD-bitmap IDE

Руководство пользователя LCD-bitmap IDE

Пошаговый workflow для новичков: проектирование, проверка и экспорт LCD-интерфейсов для измерительных сценариев.

- | | | |
|---------------------------------------|---------------------------------------|-------------------------------|
| 1. Что делает приложение | 2. Установка и запуск | 3. Открытие демо-проекта |
| 4. Основные рабочие области | 5. Построение измерительного workflow | 6. Проектирование LCD-экранов |
| 7. Привязка кнопок, tags и procedures | 8. Проверка workflow | 9. Экспорт для firmware |
| 10. Сохранение, версии и передача | 11. Устранение неполадок | |

1. Что делает приложение

LCD-bitmap IDE - offline desktop-инструмент для проектирования монохромных LCD-экранов, FSM-навигации, привязок панели управления и firmware-ready экспортов.

Используйте его для проектирования интерфейса вокруг измерительного workflow: настройка, старт, ход выполнения, просмотр результата, предупреждения и сервисные состояния.

Само приложение не выполняет физические измерения. Для реального снятия данных нужен firmware прибора, backend-сервис или hardware connector. В LCD-bitmap IDE вы проектируете и проверяете интерфейс и runtime-поведение до интеграции с целевой прошивкой.

- ❑ Создать или открыть проект.
- ❑ Спроектировать экраны и FSM-переходы.
- ❑ Проверить workflow в preview.
- ❑ Экспортировать firmware assets.

2. Установка и запуск

1. Установите Node.js 22 или новее и Git.
2. Склонируйте репозиторий и выполните `npm ci`.
3. Запустите browser-сборку через `npm run dev` или desktop-сборку через `npm run electron:dev`.
4. Desktop-сборка нужна для REST API, MCP-автоматизации, PDF-экспорта и packaging.

```
git clone https://github.com/myahlovvlad/LCD-bitmap-IDE.git
cd LCD-bitmap-IDE
npm ci
npm run electron:dev
```

3. Открытие demo-проекта

На стартовом экране выберите встроенный demo-проект или откройте `examples/universal-lcd-demo.lcdproj`.

Demo специально сделан универсальным. Он показывает, как представить измерительный интерфейс без vendor implementation files, private device identifiers и приватных каталогов экранов.

1. Откройте demo.
2. Посмотрите вкладки FSM, LCD, Tags и Runtime.
3. Не меняйте всё сразу. Измените один экран или переход, затем проверьте preview.

4. Основные рабочие области

Область	Назначение	Результат для новичка
FSM	Состояния и переходы	Понятный операторский flow
LCD	Пиксельные экраны 128x64	Читаемые screen layouts
Control panel	Привязка кнопок и событий	Предсказуемый ввод оператора
Tags	Именованные значения экранов и процедур	Traceable измерительные значения
Runtime	Интерактивная проверка flow	Проверенный workflow до firmware export

5. Построение измерительного workflow

Начинайте не с пикселей, а с состояний, которые ожидает оператор. Простой workflow обычно включает Idle, Setup, Ready, Running, Result, Error и Service.

Каждое состояние должно отвечать на один вопрос пользователя: что происходит, какое значение важно сейчас и какое действие доступно.

1. Добавьте состояния во вкладке FSM.
2. Назовите переходы событиями START, NEXT, CANCEL или RETRY.
3. Сделайте error transitions явными.
4. Проверьте workflow в Runtime Preview до детальной графики.

6. Проектирование LCD-экранов

Проектируйте экран для быстрого чтения. Текущее состояние обычно сверху, ключевое значение в центре, доступное действие ближе к нижней части.

Используйте текст, линии, прямоугольники, bitmap-слои и special elements: progress bar, indicators, checkbox/radio. Не перегружайте экран декоративными деталями.

- ☐ Текст читается на 128x64.
- ☐ Важные значения имеют единицы измерения.
- ☐ Доступное действие видно.
- ☐ Warning screens визуально отличаются.

7. Привязка кнопок, tags и procedures

Кнопки должны генерировать события, совпадающие с FSM transitions. Tags описывают измерительные значения, настройки или status flags. Procedures описывают backend-действия: старт измерения, сброс результата, сервисная процедура.

Используйте стабильные имена. Tag `sample_absorbance` проще поддерживать, чем `value1`.

8. Проверка workflow

1. Откройте Runtime.
2. Начните с initial state.
3. Нажимайте кнопки в порядке реального оператора.
4. Проверьте, что все status screens появляются в правильной последовательности.
5. Проверьте error и cancel paths, а не только успешный сценарий.

9. Экспорт для firmware

Когда workflow стабилен, экспортируйте экраны как C headers, binary frame buffers, XBM, Arduino PROGMEM, Rust embedded-graphics или ESP-IDF assets.

Перед прошивкой hardware проверьте, что целевой дисплей использует ту же resolution, page order, bit order и monochrome format.

10. Сохранение, версии и передача

- ❑ Сохраняйте .lcdproj после смысловых изменений.
- ❑ Коммитьте project files и generated assets отдельно.
- ❑ Добавляйте screenshots в review.
- ❑ Связывайте firmware artifacts с версией проекта.

11. Устранение неполадок

Симптом	Вероятная причина	Действие
Кнопка Runtime ничего не делает	Нет transition для события	Проверьте event names в FSM
Export выглядит инвертированным	Bit order целевого дисплея отличается	Проверьте byte order драйвера
Текст обрезан	Объект или шрифт слишком большой	Сократите текст или измените позицию
API connection refused	Desktop app не запущен	Запустите npm run electron:dev или packaged app